

Trabajo Práctico Especial

Autómatas, Teoría de Lenguajes y

Compiladores



Franco Morroni - 60417

Tobias Pugliano - 62180

Bernardo Zapico - 62318

Alejo Padula Morillo - 63571

Tabla de contenidos

Tabla de contenidos.....	2
Introducción.....	3
Modelo Computacional.....	4
Dominio.....	4
Lenguaje.....	4
Implementación.....	6
Frontend.....	6
Análisis Léxico.....	6
Análisis Sintáctico.....	6
Backend.....	7
Análisis Semántico.....	7
Generación de Autómatas.....	7
Generación de código.....	8
Dificultades Encontradas.....	8
Futuras Expansiones.....	9
Conclusiones.....	10
Referencias.....	11

Introducción

PenguinScript es un lenguaje que le permite al usuario crear y manipular gramáticas y lenguajes, así como realizar operaciones básicas entre estos. Y finalmente se puede obtener una visualización del autómata equivalente.

El lenguaje está pensado para ser una herramienta que ayude en el estudio o investigación de la teoría de lenguajes. Está detallado en este informe su funcionalidad, el proceso de su desarrollo y futuros planes.

Modelo Computacional

Dominio

El dominio de este proyecto se centra en la traducción de gramáticas a autómatas. En particular de gramáticas regulares con producciones lineal-derechas a autómatas finitos. Como se mencionó en la introducción, PenguinScript está pensada para facilitar el estudio de la teoría de lenguajes y como parte de esto se tiene la visualización en forma de autómatas. Para ello se optó por Graphviz[1], un software open source para visualizar gráficos, por medio de su gramática estática Dot.

Por otro lado, el alfabeto se acota a caracteres y cadenas ASCII-extendido para no tener que preocuparse por encoding.

Lenguaje

Hay múltiples elementos que componen la entrada del lenguaje. Primero están los tipos:

- Primitivos: símbolo, producción
- Compuestos: gramática, conjunto de símbolos, conjunto de producciones, lenguajes.

Las variables se definen de forma $X = Y$, donde X es el identificador de la variable e Y es un valor de alguno de los tipos compuestos especificados arriba, o una expresión que evalúe a uno de ellos.

Una gramática se define como: $\langle x, y, z, w \rangle$, donde x es un conjunto de símbolos (el alfabeto), y es un conjunto de símbolos no-terminales, z es un conjunto de producciones y w es un símbolo que pertenece a y .

Un conjunto de símbolos se define como: $\{ a_1, a_2, \dots, a_n \}$, donde los a_i deben estar compuestos por caracteres ASCII-extendido.

Un conjunto de producciones se define como: $\{ A_1 \rightarrow b_1, A_2 \rightarrow b_2, \dots, A_n \rightarrow b_n \}$. Opcionalmente alguna o varias de las clausuras podría ser de la forma $A \rightarrow b_1 \mid b_2 \mid \dots \mid b_n$, donde:

- Los A_i verifican que pertenecen a y
- Los b_i son de la forma: cD , c o λ (con c en x y D en y)

Notar que se define de esta forma una gramática regular lineal por derecha.

Para conjuntos del mismo tipo se tienen las siguientes operaciones; unión ($\cup$), intersección ($\cap$), resta ($-$).

Sea G una gramática definida, $L(G)$ representa el lenguaje de G . Se definen como operación la unión ($\cup$).

Por último están los símbolos reservados: $\{, \}, =, <, >, \rightarrow, \lambda, \cup, \cap, -, |, ' '$ (espacio), $\cdot, L, \wedge, R, \neg$. La discrepancia entre las operaciones y los símbolos reservados se elabora en la sección de Dificultades Encontradas

Implementación

Frontend

Análisis Léxico

En la etapa de análisis léxico se buscó definir el vocabulario del lenguaje. Se usaron principalmente los contextos para diferenciar el uso de ciertos caracteres para distintos tipos, entre los más importantes, para las gramáticas, los conjuntos de símbolos o producciones y para los lenguajes, y para las operaciones de esta. Cabe destacar que, muchos de los lexemas utilizados son válidos para varias situaciones, como operaciones de conjuntos/lenguajes, se buscó no abusar mucho de los contextos y redefinir muchas reglas para cada contexto y, en su lugar, tener ciertos lexemas generales que dependiendo de su uso sintáctico, funcionen para una u otra cosa.

En etapas más avanzadas del proyecto, se notó que las operaciones de conjuntos se realizaban sólo con conjuntos literales, y no con variables. Para conseguir que estas operaciones tuvieran sentido se tenía que asegurar que los identificadores de las variables existentes se guarden en una una tabla de símbolos, esto se mencionará más en detalle en la siguiente etapa, pero fue necesario agregar en las acciones de Flex la posibilidad de enviar tokens de los tipos de los identificadores cuando se encontraba que este ya existía en dicha tabla.

Análisis Sintáctico

Para esta etapa se resolvió completamente todo lo relacionado a sets. La sintaxis ya valida los tipos de los sets literales, y los id tipados validan los tipos de las variables usadas, con lo cual una expresión de sets se puede resolver directamente durante el análisis sintáctico.

Algo importante de esta etapa fue tipar los distintos ids de las variables, para el uso de operaciones de conjuntos/lenguajes con ids. Para esto se utilizó una tabla de símbolos que se podría utilizar en todas las capas del proyecto, se guardaba un id a la tabla, junto con su tipo y el valor asociado a dicho id (ej: el set, la gramática).

Para el AST generado, se utilizó una lista de **sentences** para facilitar el recorrido del mismo. Y lo mismo para los conjuntos, se los agrupó en un solo nodo, ya que sino sería muy engorroso su recorrido en la capa de Backend (se necesitaría recorrer por cada elemento).

Para los tipos de las variables usadas en las **GrammarDeinition's** se buscó validarlas en esta etapa. Por ejemplo, dado un $G = \langle \text{sigma}, N, P, S \rangle$ la semántica ya valida que sigma sea un **SymbolSet**, N sea un **SymbolSet**, P sea un **ProductionSet** (gracias a los ID tipados). Si alguna de las variables no está definida se agrega a la tabla con el tipo correspondiente y su valor, y cuando se inicialice esa variable también se va a validar que el valor sea del tipo correcto.

Backend

Para el proyecto se utilizó una implementación propia de hash set para representar los conjuntos del programa. Esta misma implementación se utilizó para tabla de símbolos, armando un wrapper sobre la misma para facilitar su uso. Como función de hasheo se usó una implementación de murmurhash3[2] de 32 bits.

Análisis Semántico

Debido a la validación que se consiguió en la etapa de análisis sintáctico, en esta etapa solo resta validar que las definiciones de gramáticas son correctas. Esto implica verificar que todas la variables referenciadas existen, y que estas en conjunto conforman una gramática tipo 3 válida (podrían ser válidas para una definición de gramática y no para otra).

Para que la gramática sea considerada válida se debe cumplir que la intersección del conjunto de símbolos terminales y no terminales sea vacía, que el conjunto de no terminales contenga el símbolo inicial especificado y que el conjunto de producciones sea lineal-derecho.

Generación de Autómatas

Una vez validadas todas las gramáticas se generan los autómatas para cada gramática y lenguaje. Los autómatas generados son NDFA- λ y pueden tener estados inalcanzables. Los

lenguajes pueden referenciar a una gramática o a otro lenguaje previamente definido, y puede operar sobre estos para formar un nuevo lenguaje. La única operación válida es la unión.

Generación de código

Una vez creados todos los autómatas se procede a generar el código dot que representa a cada uno. El output se hace por salida estándar.

Dificultades Encontradas

- El output del compilador era originalmente en el lenguaje Mermaid, pero este lenguaje tenía problemas para mostrar loops (transiciones de un nodo a sí mismo) con lo cual se terminó optando por usar Dot.
- Para la generación del autómata surgió el problema de que se requería agregar estados nuevos al mismo. Esto resultó difícil ya que los estados agregados deben ser distintos a los ya definidos por el usuario en la gramática. La solución que se utilizó fue agregar un prefijo a los estados del usuario y no usar ese prefijo para los estados nuevos, lo cual asegura que no vaya a haber colisión de nombres.
- Las operaciones de lenguajes aparte de la unión eran considerablemente más complicadas de implementar, con lo cual debido a limitaciones de tiempo se optó por removerlas del lenguaje.

Futuras Expansiones

Entre las futuras expansiones previstas para próximas fases, consideramos la implementación de nuevas operaciones y funcionalidades para nuestro lenguaje. Estas mejoras están orientadas a potenciar la experiencia del usuario y garantizar la adaptabilidad del lenguaje a diferentes contextos. Entre estas consideramos:

- Generación de autómatas para todas las operaciones de lenguajes
- Poder generar el DFA mínimo y no solo el NDFA- λ
- Aceptar gramáticas lineal-izquierdas
- Aceptar más tipos de gramáticas, y no solo de tipo 3
- Devolver imágenes en vez de código Dot para mejor User Experience
- Analizar si una cadena es aceptada o rechazada por el lenguaje del autómata

Conclusiones

Si bien el proyecto tiene potencial para crecer aún más tanto en funcionalidad como calidad de código, se considera satisfactoria su funcionalidad en esta instancia. Se espera poder desarrollar adecuadamente las futuras expansiones para crear una herramienta open source con calidad para que se adopte comúnmente.

Referencias

[1] Graphviz (28/09/2024). DOT Language. <https://graphviz.org/doc/info/lang.html>

[2] Wolkykim (14/05/2024) . qlibc. <https://github.com/wolkykim/qlibc>